

Behavioral Interview Prep – Authorization Service (C++)

You built the **Authorization Service** in C++ using `librdkafka` and `libpqxx`.

Use the **STAR format**: Situation → Task → Action → Result

Your Role – Quick Intro

Use this when asked: "What did YOU specifically work on?"

"I built the **Authorization Service** – a high-performance C++ microservice that validates every payment before it can proceed. It consumes payment requests from a Kafka topic, checks user existence, verifies payment intent, validates the payment method, checks for card expiry, and confirms the user has sufficient funds – all using a single optimized SQL query. If authorized, it publishes to `payment_authorized`; if not, to `payment_failed`. I also built the **Thread Pool**, **DB Connection Pool**, and **DLQ error routing** from scratch."

★ Story 1 – The Floating-Point Bug (A Real Bug You Fixed)

Question type: "Tell me about a bug you found and fixed." / "Tell me about attention to detail."

STAR:

- **S:** During testing, some payment requests were being incorrectly rejected with `"Payment amount mismatch with intent"` even though the amounts looked correct on screen.
- **T:** Find and fix why the amount comparison between the Kafka event and the database was failing.
- **A:** Traced it to a floating-point formatting issue. The amount from the Kafka event (a `double`) was being converted to a string using `std::to_string()`, which uses `%g`-style formatting – it could produce `"13644.959999"` instead of `"13644.96"`. The DB stored it as `numeric(15,2)` which always rounded to 2 decimal places. The strings never matched. I fixed it by replacing `std::to_string()` with `std::fixed + std::setprecision(2)` via `std::ostringstream`, which correctly rounds to 2 decimal places.

- **R:** Amount comparisons became perfectly reliable. No more false rejections. This was a subtle but critical bug — in a payment system, a false rejection costs real money.

💡 If they ask: "Show me the fix." — `oss << std::fixed << std::setprecision(2)`
`<< event.amount;`

★ Story 2 – Consumer Thread Blocking Problem (Design Decision)

Question type: "Tell me about a performance optimization." / "Tell me about a design decision."

STAR:

- **S:** The Kafka consumer runs on a single thread. Each incoming payment message required a DB query to validate it — which takes time. If the consumer thread was blocked waiting for the DB, it couldn't pick up the next Kafka message — creating a growing backlog.
 - **T:** Design the service to process messages concurrently without blocking the consumer thread.
 - **A:** I implemented a **Thread Pool** from scratch in C++ with 100 worker threads (configurable via env var). When a Kafka message arrives, the consumer thread immediately copies the payload and uses `enqueue_detach()` to hand it off to the thread pool — freeing itself instantly to consume the next message. The 100 worker threads handle DB queries and Kafka publishing in parallel.
 - **R:** The consumer thread was never blocked by business logic. The service could handle many concurrent authorizations simultaneously, contributing to our overall 10,000 RPS target.
-

★ Story 3 – DB Connection Pool (Solving Resource Exhaustion)

Question type: "Tell me about a scalability challenge." / "Tell me about a resource management problem."

STAR:

- **S:** With 100 worker threads all running simultaneously, each needing a PostgreSQL connection, naively opening a new DB connection per thread would exhaust PostgreSQL's connection limit and crash the service.
- **T:** Manage database connections efficiently so threads share a fixed pool rather than creating unlimited connections.
- **A:** Built a **Connection Pool** in C++ using a `std::vector` of pre-created `pqxx::connection` objects, protected by a `std::mutex` and `std::condition_variable`. When a worker

thread needs a connection, it takes one from the pool. If none are available, it **blocks and waits** instead of crashing. A `ConnectionGuard` RAll wrapper ensures the connection is always returned to the pool when the thread finishes – even if an exception is thrown.

- **R:** DB connections were capped at 100 (configurable), preventing resource exhaustion. The RAll pattern meant zero connection leaks, even in error paths.
-

★ Story 4 – Dead Letter Queue Routing (Failure Handling)

Question type: "How did you handle failures in your service?" / "Tell me about fault tolerance."

STAR:

- **S:** In production, any worker thread could fail – due to a malformed JSON message, a DB timeout, or a network error. If an exception wasn't handled properly, the message would be silently dropped and the transaction would never complete.
 - **T:** Ensure no payment message is silently lost, even when the authorization worker throws an exception.
 - **A:** Wrapped the entire worker logic in a `try-catch`. On any exception, instead of just logging and moving on, I published the original raw payload to `payment_initiated.DLT` (Dead Letter Topic). I also added a nested try-catch around the DLT publish itself – because even publishing to the DLT can fail – logging a clear error if that happened too.
 - **R:** Failed messages were never silently dropped. The operations team could monitor the DLT topic and re-process or investigate failed payments manually. This is exactly how production payment systems handle failures.
-

★ Story 5 – JSON Schema Mismatch (Cross-Team Integration Problem)

Question type: "Tell me about a collaboration challenge." / "Tell me about working with other teams."

STAR:

- **S:** The API Gateway (Java/Spring Boot) published Kafka messages using camelCase field names (`transactionId`, `userId`). My C++ Authorization Service was reading snake_case (`transaction_id`, `user_id`). Initially, all fields were coming in as empty strings – the service was authorizing or rejecting payments based on blank data.

- **T:** Make the C++ service robust to both naming conventions without requiring the Java team to change their code.
 - **A:** Modified the `from_json` parser in `PaymentModels.h` to use `j.value("transaction_id", j.value("transactionId", ""))` – checking for both naming conventions. Same pattern for all fields. This made the C++ service accept messages from any service regardless of naming style.
 - **R:** Integration between the Java API Gateway and C++ Authorization Service worked seamlessly without either team changing their core logic. This pattern also made the service more resilient to future schema changes.
-

★ Story 6 – Single Optimized SQL Query (Performance Thinking)

Question type: "Tell me about how you optimized something." / "Tell me about your attention to performance."

STAR:

- **S:** The authorization logic needed to check 5 things: does the user exist, does the payment intent exist, does the payment method belong to the user, is the card expired, and does the user have sufficient funds. A naive implementation would make 5 separate DB queries per payment.
 - **T:** Minimize DB round trips because at 10,000 RPS, 5 queries per payment = 50,000 DB queries per second – completely unsustainable.
 - **A:** Collapsed all checks into a **single optimized LEFT JOIN query** across 4 tables (`users`, `payment_intents`, `payment_methods`, `accounts`) using parameterized placeholders (`$1`, `$2`, `$3`) for SQL injection safety. A single query returns all needed data in one round trip. The service then checks the result row sequentially for each failure condition.
 - **R:** Each authorization required only 1-2 DB queries (the second only runs for CARD type to check expiry date). This dramatically reduced DB load and latency per transaction.
-



Your Role – One-liner Summary

"I owned the Authorization Service end-to-end – the C++ service, its Thread Pool, Connection Pool, Kafka consumer/producer integration, the core authorization SQL logic, JSON schema handling, and DLQ error routing."

? Quick Q&A Cheat Sheet

Question	Key Answer
"What language did you use and why C++?"	High-performance validation engine; C++ gives low latency and full control over threads and memory – critical for a bottleneck service at 10k RPS
"What's librdkafka?"	C/C++ client library for Apache Kafka – used to consume from <code>payment_initiated</code> and publish to <code>payment_authorized</code> / <code>payment_failed</code>
"What's libpqxx?"	C++ client library for PostgreSQL – used to run SQL queries against the user/account database
"What checks does the service do?"	User exists → Payment intent matches → Amount matches → Payment method belongs to user → Card not expired → Sufficient funds
"What happens if authorization fails?"	Publishes to <code>payment_failed</code> topic with a specific reason string (e.g., "Insufficient funds", "Payment method expired")
"What happens if the service crashes?"	Kafka retains the message. After restart, consumer reads from <code>auto.offset.reset=earliest</code> and replays unprocessed messages
"What was the hardest part?"	The floating-point precision bug – subtle, silent, and dangerous in a financial system
"What would you improve?"	Add metrics (Prometheus counters) per failure reason – e.g., how many rejections per day are "Insufficient funds" vs "Expired card"